

PYTHON MASTER NOTES

PART 5

Memory, Iteration, OOP & Exceptions

Deeper Python behaviour for college study and technical interviews

Chapter	What it covers
1. References & Memory	names, objects, <code>id()</code> , aliasing, mutability, shallow/deep copy, <code>[[0]*3]*3</code> , argument passing, scope
2. Iterables & Iterators	<code>iter()</code> , <code>next()</code> , <code>StopIteration</code> , the <code>for</code> loop internals, custom iterators
3. Generators	<code>yield</code> , generator expressions, lazy evaluation, <code>yield from</code> , generator vs list
4. Exception Handling	<code>try/except/else/finally</code> , <code>raise</code> , custom exceptions, built-ins, best practices
5. OOP	classes, methods, encapsulation, inheritance, polymorphism, composition, <code>super()</code> , MRO, dunder methods
6-11. Practice & Reference	diagram gallery, terminology cheat sheet, 120 questions with full solutions, common traps, revision sheet

Colour key for the boxes used throughout these notes:

CONCEPT: Blue = core idea

The definition or rule you must be able to state.

MENTAL MODEL: Purple = mental model

A picture or analogy that makes the rule stick.

TIP: Green = good practice

What to do in real code.

COMMON TRAP: Red = trap

A classic bug or exam trick.

INTERVIEW: Amber = interview angle

How to phrase the answer out loud.

Contents

1. References & Memory	4
1.1 Variables are names, not boxes	4
1.2 Objects: identity, type, value	4
1.3 Assignment and aliasing	5
1.4 Mutable vs immutable	5
1.5 Copying: alias, shallow copy, deep copy	6
1.5.1 The <code>[[0]*3]*3</code> trap	7
1.6 Function arguments: pass by object reference	7
1.7 Mutable default arguments	8
1.8 Reference counting and garbage collection	8
1.9 Scope refresher: LEGB, global, nonlocal, closures	9
2. Iterables & Iterators	10
2.1 The two roles	10
2.2 <code>iter()</code> , <code>next()</code> , <code>StopIteration</code>	10
2.3 What a for loop really does	10
2.4 One-shot behaviour and ranges	11
2.5 Traps while iterating	11
2.6 Custom iterators	11
3. Generators	13
3.1 Generator functions and <code>yield</code>	13
3.2 Lazy evaluation and memory	13
3.3 Generator expressions	14
3.4 <code>yield</code> vs <code>return</code>	14
3.5 Generator vs list	14
3.6 <code>yield from</code> , return values, <code>send()</code>	14
4. Exception Handling	16
4.1 What an exception is	16
4.2 <code>try / except / else / finally</code>	16
4.3 Handling several exceptions	16
4.4 <code>raise</code> , re-raise, chaining	17
4.5 <code>finally</code> details and the return trap	17
4.6 Custom exceptions	18
4.7 Common built-in exceptions	18
4.8 Best practices	19
5. Object-Oriented Programming	20
5.1 Class, object, <code>self</code> , <code>__init__</code>	20

5.2 Instance variables vs class variables	20
5.3 Instance, class and static methods	21
5.4 Encapsulation	22
5.5 Abstraction	22
5.6 Inheritance, overriding, super()	23
5.7 Polymorphism and duck typing	23
5.8 Method overloading in Python	23
5.9 Composition	24
5.10 Multiple inheritance and MRO	24
5.11 Dunder (magic) methods	25
6. Diagram Gallery	27
6.1 Mutation vs rebinding	27
6.2 The iterator protocol as a conversation	27
6.3 List vs generator in memory	27
6.4 try / except / else / finally flow	27
6.5 Attribute lookup	28
6.6 Bound methods	28
6.7 C3 merge, step by step	28
6.8 Closures and cells	28
6.9 Everything is an object	28
6.10 Exception propagation with finally	29
7. Terminology Cheat Sheet	30
8. Question Bank	32
8A. 40 Conceptual Questions	32
8B. 40 Output Questions (predict the output)	33
8C. 40 Interview Questions	37
9. Complete Solutions	39
9A. Conceptual Answers	39
9B. Output Answers	41
9C. Interview Answers	42
10. Common Traps	46
11. Revision Sheet	47
References & memory	47
Iterables, iterators, generators	47
Exceptions	47
OOP	47
30-second self-test	48

1. References & Memory

This is the single most important chapter for interviews. Almost every "weird Python output" question is really a question about **names, objects, and references**. Read it slowly and run every example yourself.

1.1 Variables are names, not boxes

In C++, `int x = 10;` reserves a box in memory called `x` that holds 10. In Python, `x = 10` does something different: it makes (or finds) an **int object** holding 10 and **binds the name x** to it. Assignment never copies an object; it only makes a name point at an object.

DIAGRAM: names point to objects

```
x = [1, 2, 3]
y = x
```

NAMES (in a namespace)

OBJECTS (in heap memory)

```
+-----+
| x |-----+
+-----+
+-----+
| y |-----+
+-----+
+-----+> | list [1, 2, 3]
+-----+ | id = 0x7f3a...
+-----+
```

Two names, ONE object. `y = x` did not copy the list.

MENTAL MODEL: Sticky notes and things

Think of names as **sticky notes** and objects as **things**. You can stick many notes on one thing. Writing `y = x` sticks a second note on the same thing. **Rebinding** (`x = something_else`) peels a note off and sticks it on a different thing. The thing itself is untouched.

1.2 Objects: identity, type, value

Every Python object has exactly three properties:

Property	Meaning	How to get it	Can it change?
Identity	Unique ID while the object lives (in CPython it is the memory address)	<code>id(obj)</code>	Never
Type	The class the object is an instance of	<code>type(obj)</code>	Practically never
Value	The data the object holds	the object itself	Only if the object is mutable

```
x = [1, 2]
print(type(x))      # <class 'list'>
print(id(x))        # some integer, e.g. 140234567890
y = x
print(id(x) == id(y)) # True -> same object
```

== versus is

Operator	Question it answers	Compares
<code>a == b</code>	Do a and b have equal values ?	value (calls <code>__eq__</code>)
<code>a is b</code>	Are a and b the same object ?	identity (<code>id(a) == id(b)</code>)

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b)  # True    equal values
print(a is b)  # False   two different list objects
print(a is c)  # True    same object
```

COMMON TRAP: Never use `is` to compare numbers or strings

CPython caches small ints (-5 to 256) and may reuse identical string constants. So `is` sometimes *appears* to work for equal numbers/strings and then fails on bigger values or computed values. This is an **implementation detail**, not a language rule.

- Use `==` for values.
- Use `is` only for singletons: `x is None`, `x is True`, `x is False`.

1.3 Assignment and aliasing

Aliasing means two or more names refer to the same object. It is created by plain assignment, by passing an object into a function, by storing it in two containers, and by `[x] * n`.

There are only two things you can do to an object through a name, and mixing them up causes most bugs:

Operation	Example	What happens	Visible via other names?
Rebinding	<code>a = [9]</code>	Name <code>a</code> now points to a different object	No
Mutating	<code>a.append(4)</code> , <code>a[0] = 1</code>	The object itself changes in place	Yes , everyone holding a reference sees it

```
a = [1, 2, 3]
b = a          # alias: b is another name for the same list
b.append(4)    # MUTATION -> visible through a
print(a)       # [1, 2, 3, 4]
b = [9]        # REBINDING -> b now points elsewhere; a is unaffected
print(a, b)    # [1, 2, 3, 4] [9]

x = y = []     # chained assignment: ONE list, two names
x.append(1)
print(y)       # [1]
```

1.4 Mutable vs immutable

Immutable (object cannot change)	Mutable (object can change in place)
<code>int</code> , <code>float</code> , <code>complex</code> , <code>bool</code> , <code>str</code> , <code>bytes</code> , <code>tuple</code> , <code>frozenset</code> , <code>None</code>	<code>list</code> , <code>dict</code> , <code>set</code> , <code>bytearray</code> , most user-defined class instances

Immutable does not mean "the variable can never change". It means the *object* can never change. Operations like `s += "!"` build a **new** object and rebind the name.

```
s = "hi"
print(id(s))    # e.g. 1001
s += "!"        # creates a NEW str, rebinds s
print(id(s))    # different id

L = [1]
print(id(L))    # e.g. 2001
L += [2]        # list.__iadd__ mutates IN PLACE
print(id(L))    # SAME id
L = L + [3]     # builds a NEW list, rebinds L
print(id(L))    # different id
```

COMMON TRAP: A tuple can "contain" something mutable

The tuple's **references** are frozen, but the objects they point to may still be mutable.

```
t = ([1, 2], 3)
t[0].append(99) # fine: we mutate the list, not the tuple
print(t)        # ([1, 2, 99], 3)
try:
    t[0] += [5]  # list is extended FIRST, then t[0] = ... fails
except TypeError:
    print("TypeError")
print(t)        # ([1, 2, 99, 5], 3) <- yet the list changed!
```

CONCEPT: Why mutability matters: hashing

Dictionary keys and set elements must be **hashable**. Immutable objects (whose contents are hashable) are hashable; lists, dicts and sets are not. A mutable key could change its hash after insertion and become unfindable, so Python forbids it.

1.5 Copying: alias, shallow copy, deep copy

DIAGRAM: three levels of copying

```
a = [[1, 2], [3, 4]]
```

(1) ALIAS b = a
a ---+
 +--> [p0, p1] --> [1, 2] and [3, 4]
b ---+ (one outer list, two names)

(2) SHALLOW COPY b = a.copy() new OUTER list, SAME inner lists
a ----> [p0, p1]
 | |
 v v
 [1,2] [3,4] <- shared inner objects
 ^ ^
 | |
b ----> [p0, p1]

(3) DEEP COPY b = copy.deepcopy(a) everything duplicated recursively
a ----> [p0, p1] ----> [1,2] [3,4]
b ----> [q0, q1] ----> [1,2]' [3,4]'

Technique	Kind	Notes
b = a	Alias (no copy)	Same object, two names
a[:]	Shallow	Slice of a list
a.copy()	Shallow	Available on list, dict, set
list(a), dict(a), set(a)	Shallow	Constructors build a new container
copy.copy(a)	Shallow	Works for almost any object
copy.deepcopy(a)	Deep	Recursive; uses a memo dict so cycles and shared sub-objects are handled
[row[:] for row in grid]	Manual 2-level copy	Common and fast for 2D lists of numbers

```
import copy
a = [[1, 2], [3, 4]]
s = copy.copy(a)                      # shallow
d = copy.deepcopy(a)                  # deep

a[0].append(99)                      # mutate an INNER list
print(s)    # [[1, 2, 99], [3, 4]]    shared inner list changed too
print(d)    # [[1, 2], [3, 4]]        independent

a.append([5])                        # change the OUTER list
print(s)    # [[1, 2, 99], [3, 4]]    outer lists are separate, s is unchanged
```

- A shallow copy of a **flat list of immutables** ([1, 2, 3]) is effectively independent, because there is nothing mutable inside to share.
- **deepcopy** is slower and copies everything; use it only when you truly have nested mutable data.
- **copy.copy(t)** on an immutable tuple of immutables may simply return the same object; copying it is pointless.
- Custom classes can control copying by defining **__copy__** and **__deepcopy__**.

1.5.1 The `[[0]*3]*3` trap

`[x] * n` **repeats the reference**, it does not copy the object. That is harmless when `x` is immutable (`[[0] * 5]` is perfectly fine) and a disaster when `x` is mutable.

DIAGRAM: row aliasing

<code>m = [[0]*3]*3</code>	<code>m = [[0]*3 for _ in range(3)]</code>
<code>m --> [p, p, p]</code>	<code>m --> [p, p, p]</code>
<code> </code>	<code> </code>
<code> +--+--+</code>	<code> v v v</code>
<code> </code>	<code> [0,0,0] [0,0,0] [0,0,0]</code>
<code> v</code>	<code>three DIFFERENT lists</code>
<code> [0, 0, 0]</code>	
<code>ONE list, referenced 3 times</code>	

```
bad = [[0] * 3] * 3
bad[0][0] = 1
print(bad)      # [[1, 0, 0], [1, 0, 0], [1, 0, 0]]  all rows changed!

good = [[0] * 3 for _ in range(3)]
good[0][0] = 1
print(good)     # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Same disease: `[[[]] * 3, [{}]] * 3, [set()] * 3`. Same cure: a comprehension that builds a fresh object each time.

1.6 Function arguments: pass by object reference

Python is neither "pass by value" nor "pass by reference" in the C++ sense. It is **pass by object reference** (also called *call by sharing*): the parameter is a **new local name bound to the same object** the caller passed. So:

- **Mutating** the object inside the function is visible to the caller (same object).
- **Rebinding** the parameter inside the function is invisible to the caller (only the local name moves).
- Immutable objects can only ever be rebound, so the caller never sees a change.

DIAGRAM: argument passing

<code>nums = [1]</code>	<code>modify(nums)</code>	<code>inside modify(lst):</code>
<code>caller nums ---+</code>	<code>lst.append(100)</code>	<code>-> mutates shared object</code>
<code> +---> [1]</code>		
<code>callee lst ---+</code>	<code>lst = [0]</code>	<code>-> lst now --> [0] (new object)</code>
		<code>nums still --> [1, 100]</code>

```
def modify(lst):
    lst.append(100)      # mutation: caller sees it

def rebind(lst):
    lst = [0]           # rebinding: only the local name changes

def inc(n):
    n += 1              # int is immutable: new object, local rebinding

nums = [1]
k = 5
modify(nums)
rebind(nums)
inc(k)
print(nums, k)         # [1, 100] 5
```

COMMON TRAP: += vs = ... + inside a function

```
def f1(lst):
    lst += [1]          # in-place: caller's list CHANGES

def f2(lst):
    lst = lst + [1]     # new list bound to local name: caller's list unchanged

a = []; f1(a); print(a)  # [1]
b = []; f2(b); print(b)  # []
```

1.7 Mutable default arguments

Default values are evaluated **once**, when the **def** statement runs, and stored on the function object. A mutable default is therefore shared by every call that does not supply the argument.

```
def add_item(x, bucket=[]):      # the [] is created ONCE
    bucket.append(x)
    return bucket

print(add_item(1))              # [1]
print(add_item(2))              # [1, 2]  surprise: same list reused
print(add_item.__defaults__)    # ([1, 2],)

def add_item_ok(x, bucket=None): # standard fix: None sentinel
    if bucket is None:
        bucket = []
    bucket.append(x)
    return bucket
```

INTERVIEW: Interview follow-up: is a mutable default ever useful?

Yes, deliberately, as a cheap per-function cache or memo (**def fib(n, memo={})**). Say so, and add that it is a trick that should be commented, because readers will assume it is a bug.

1.8 Reference counting and garbage collection

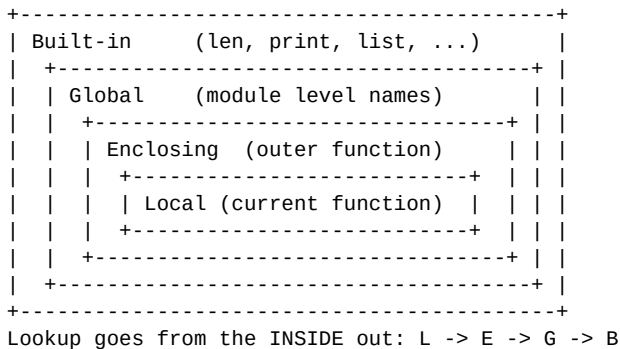
- CPython stores a **reference count** on every object. When it reaches 0, the object is freed immediately.
- **del x** removes the **name x**, not necessarily the object. The object dies only when nothing else refers to it.
- Objects that refer to each other in a cycle never reach 0 by counting alone, so CPython also runs a **cyclic garbage collector** (gc module).
- **sys.getrefcount(obj)** reports one more than you expect, because passing **obj** to the function creates a temporary reference.

```
import sys
a = [1]
print(sys.getrefcount(a))  # 2 (name a + the function argument)
b = a
print(sys.getrefcount(a))  # 3
del b                      # removes the name b only
print(sys.getrefcount(a))  # 2
```

1.9 Scope refresher: LEGB, global, nonlocal, closures

When Python meets a name it searches four scopes in order: **Local** (current function), **Enclosing** (outer functions), **Global** (module), **Built-in**.

DIAGRAM: LEGB



- **Reading** a name finds it by LEGB. **Assigning** to a name inside a function makes it **local** for the entire function, decided at compile time.
- That is why reading a global and then assigning it in the same function raises **UnboundLocalError**.
- **global x** lets a function rebind a module-level name. **nonlocal x** lets an inner function rebind a name in the enclosing function.
- Mutating a global list/dict needs no **global** statement (no rebinding happens).

```
x = 10
def f():
    print(x)          # UnboundLocalError: x is local because of the assignment below
    x = 5
# f()

def make_counter():
    c = 0
    def inc():
        nonlocal c    # without this: UnboundLocalError
        c += 1
        return c
    return inc         # inc is a CLOSURE: it remembers c

counter = make_counter()
print(counter(), counter(), counter())    # 1 2 3
```

COMMON TRAP: Late binding in closures and lambdas

A closure looks up the variable **when it is called**, not when it was defined.

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])    # [2, 2, 2] every lambda sees the final i

fs = [lambda i=i: i for i in range(3)] # default arg captures the value NOW
print([f() for f in fs])    # [0, 1, 2]
```

COMMON TRAP: Shadowing built-ins

list = [1, 2] or **sum** = 0 rebinds a global name that hides the built-in. Later **list(range(3))** fails with **TypeError: 'list' object is not callable**. Never name variables **list**, **dict**, **str**, **sum**, **max**, **min**, **id**, **type**, **input**.

INTERVIEW: Interview: "Is Python pass by value or pass by reference?"

Model answer: "Neither exactly. It is pass by object reference. The function receives a new name bound to the same object. If the function mutates a mutable object the caller sees it; if it rebinds the parameter, the caller does not." Then give the **modify** / **rebind** example from 1.6.

2. Iterables & Iterators

2.1 The two roles

Term	Definition	Protocol methods	Re-usable?
Iterable	Any object you can loop over. <code>iter(obj)</code> gives you an iterator.	<code>__iter__()</code> (returns an iterator)	Yes: every <code>iter()</code> call gives a fresh iterator
Iterator	An object that produces the items one at a time and remembers its position.	<code>__next__()</code> and <code>__iter__()</code> (returns itself)	No : once exhausted it stays exhausted

Iterable but NOT an iterator	Iterators
<code>list</code> , <code>tuple</code> , <code>str</code> , <code>dict</code> , <code>set</code> , <code>range</code> , <code>frozenset</code> , <code>bytes</code>	<code>iter(x)</code> , generators, file objects, <code>map</code> , <code>filter</code> , <code>zip</code> , <code>enumerate</code> , <code>reversed(...)</code> , <code>itertools.*</code> results

MENTAL MODEL: Book and bookmark

An **iterable** is a book: you can start reading as many times as you like. An **iterator** is a bookmark: it knows one position and only moves forward. `iter(book)` hands you a fresh bookmark. `next(bookmark)` reads the next page.

DIAGRAM: iterable vs iterator

```
ITERABLE (list)                                ITERATOR (list_iterator)

[10, 20, 30]  ----- iter() ----->  +-----+
                                           | source  : the list |
                                           | position: 0      |
                                           +-----+

next(it) -> 10    (position 1)
next(it) -> 20    (position 2)
next(it) -> 30    (position 3)
next(it) -> raises StopIteration (no more items)
```

2.2 `iter()`, `next()`, `StopIteration`

```
nums = [10, 20, 30]
it = iter(nums)          # get an iterator
print(next(it))          # 10
print(next(it))          # 20
print(next(it))          # 30
# print(next(it))        # raises StopIteration
print(next(it, "done"))  # 'done' <- a default suppresses StopIteration
print(iter(it) is it)    # True   an iterator's __iter__ returns itself
```

- **StopIteration** is not an error in the usual sense: it is the **signal** that says "no more items".
- Once an iterator raised **StopIteration**, it must keep raising it. It cannot restart.

2.3 What a for loop really does

```
for x in iterable:
    body(x)

# is roughly the same as:
it = iter(iterable)          # step 1: ask for an iterator
while True:
    try:
        x = next(it)         # step 2: fetch next item
    except StopIteration:     # step 3: loop ends quietly
        break
    body(x)
```

Everything that consumes a sequence uses the same machinery: `list(x)`, `sum(x)`, `sorted(x)`, `min/max`, `in`, tuple unpacking, `*` unpacking, `zip`, `enumerate`, `all/any`, `"".join(x)`. The loop's `else` block runs when the iterator is

exhausted **without** a **break**.

2.4 One-shot behaviour and ranges

```
it = iter([1, 2, 3])
print(list(it))      # [1, 2, 3]
print(list(it))      # []          already exhausted

r = range(3)
print(list(r), list(r)) # [0, 1, 2] [0, 1, 2]
# next(r)              # TypeError: 'range' object is not an iterator
print(len(range(10**12)), 5 in range(10)) # 1000000000000 True (lazy, 0(1))

m = map(str, [1, 2])
print(list(m), list(m)) # ['1', '2'] [] map objects are iterators
```

2.5 Traps while iterating

COMMON TRAP: Modifying a list while looping over it

The loop keeps an **index** internally. Removing items shifts the rest left, so the loop skips elements.

```
nums = [2, 4, 6, 8]
for x in nums:
    if x % 2 == 0:
        nums.remove(x)
print(nums)          # [4, 8] not []!

nums = [2, 4, 6, 8]
nums = [x for x in nums if x % 2 != 0] # build a new list -> []
for x in nums[:]:
    # or iterate over a COPY while modifying the original
    pass
```

COMMON TRAP: Changing a dict's size while iterating

```
d = {"a": 1, "b": 2}
# for k in d: del d[k]      # RuntimeError: dictionary changed size during iteration
for k in list(d):          # iterate over a snapshot of the keys
    del d[k]
print(d)                   # {}
```

2.6 Custom iterators

To make your own iterator, write a class with `__iter__` (return `self`) and `__next__` (return the next value or raise `StopIteration`).

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self          # an iterator returns itself

    def __next__(self):
        if self.current <= 0:
            raise StopIteration # signal: finished
        value = self.current
        self.current -= 1
        return value

for n in Countdown(3):
    print(n, end=" ")        # 3 2 1
print()

c = Countdown(2)
print(list(c), list(c))     # [2, 1] [] one-shot
```

If you want something that can be looped over **many times**, separate the two roles: the **iterable** creates a **fresh iterator** each time.

```
class CountdownIterable:          # iterable (NOT an iterator)
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return Countdown(self.start)  # new iterator on every loop

r = CountdownIterable(2)
print(list(r), list(r))             # [2, 1] [2, 1]
```

TIP: Shortcut: let a generator do the work

```
class CountdownIterable2:
    def __init__(self, start):
        self.start = start
    def __iter__(self):              # generator function -> returns a fresh generator
        n = self.start
        while n > 0:
            yield n
            n -= 1
```

The two-argument form iter(callable, sentinel)

```
data = [3, 2, 0, 5]
it = iter(data)
print(list(iter(lambda: next(it), 0)))  # [3, 2] stops when the call returns 0
# classic use: for line in iter(f.readline, ""): ...
```

INTERVIEW: Interview: iterable vs iterator in one breath

"An iterable has `__iter__` and hands out an iterator; an iterator has `__next__` and `__iter__` returning itself. Lists are iterables, not iterators. Generators, files and **map/zip/filter/enumerate** are iterators, so they can be consumed only once."

3. Generators

3.1 Generator functions and yield

A function that contains **yield** is a **generator function**. Calling it does **not** run the body; it returns a **generator object** (which is an iterator). Each **next()** runs the body until the next **yield**, hands out that value, and **pauses**, keeping all local variables alive.

```
def count_up(n):
    i = 1
    while i <= n:
        yield i          # hand out i, pause here
        i += 1

g = count_up(3)
print(type(g))          # <class 'generator'>
print(next(g))          # 1
print(next(g))          # 2
print(list(g))          # [3] the rest
```

```
def demo():
    print("step 1")
    yield "A"
    print("step 2")
    yield "B"
    print("end")

g = demo()               # prints NOTHING: body has not started
print(next(g))           # step 1 then A
print(next(g))           # step 2 then B
# next(g)                # prints 'end', then raises StopIteration
```

DIAGRAM: generator lifecycle

<code>g = demo()</code>	-> generator object created, body NOT started	[CREATED]
<code>next(g) #1</code>	-> run to 1st yield, return "A", freeze	[SUSPENDED]
<code>next(g) #2</code>	-> resume just after 1st yield, run to 2nd yield	[SUSPENDED]
<code>next(g) #3</code>	-> resume, function ends -> StopIteration	[CLOSED]

Locals (i, n, ...) survive while SUSPENDED. That is the whole trick.

3.2 Lazy evaluation and memory

Lazy evaluation means a value is computed only when someone asks for it. A generator never builds the whole sequence; it keeps just its current state (a few hundred bytes), no matter how many values it can produce.

```
import sys
big_list = [x * x for x in range(1_000_000)] # builds all 1,000,000 values now
big_gen = (x * x for x in range(1_000_000)) # builds nothing yet
print(sys.getsizeof(big_list)) # roughly 8,000,000 bytes (list of pointers; varies)
print(sys.getsizeof(big_gen)) # a couple of hundred bytes, constant
```

Because generators are lazy they can also be **infinite**, and side effects (prints, file reads) happen at consumption time, not at creation time.

```
from itertools import islice

def fib():
    a, b = 0, 1
    while True:          # infinite generator
        yield a
        a, b = b, a + b

print(list(islice(fib(), 8))) # [0, 1, 1, 2, 3, 5, 8, 13]
```

3.3 Generator expressions

```
squares_list = [x * x for x in range(5)]    # list comprehension -> list
squares_gen  = (x * x for x in range(5))    # generator expression -> generator

print(sum(x * x for x in range(1000)))      # extra brackets not needed as sole argument
nums = range(1, 11)
evens  = (n for n in nums if n % 2 == 0)    # pipeline: each stage lazy
squares = (n * n for n in evens)
print(sum(squares))                        # 220 (4+16+36+64+100)
```

COMMON TRAP: A generator expression can only be consumed once, and it reads variables late

```
g = (x * x for x in range(3))
print(sum(g), sum(g))    # 5 0    second sum: generator already exhausted

x = 1
g = (x * i for i in range(3))
x = 10
print(list(g))           # [0, 10, 20]    # x is looked up when the generator RUNS
```

3.4 yield vs return

	return	yield
Effect	Ends the function for good	Pauses the function; it can resume later
Values produced	One (a single object or tuple)	Many, one per <code>next()</code>
Local variables	Destroyed	Preserved between resumes
Calling the function	Runs the body immediately	Returns a generator object; body runs on <code>next()</code>
Each call	Starts fresh	Each generator object has its own independent state
<code>return v</code> inside a generator	n/a	Ends the generator; <code>v</code> is stored in <code>StopIteration.value</code>

3.5 Generator vs list

Question	List	Generator
Memory	Holds all items: $O(n)$	Holds only state: $O(1)$
Start-up cost	Computes everything first	Instant; computes on demand
Indexing / <code>len()</code> / slicing	Yes	No
Iterate twice	Yes	No (recreate or use <code>itertools.tee</code>)
Infinite sequence	Impossible	Natural
Best for	Small data, repeated access, random access	Large/streaming data, pipelines, one pass

3.6 yield from, return values, send()

```
def chain_two(a, b):
    yield from a    # delegate to another iterable
    yield from b
print(list(chain_two([1, 2], "ab")))    # [1, 2, 'a', 'b']

def sub():
    yield 1
    return "done"    # becomes StopIteration.value

def main():
    result = yield from sub()    # yield from evaluates to sub's return value
    print("sub returned", result)

print(list(main()))    # prints 'sub returned done' then [1]
```

```
def echo():
    while True:
        received = yield          # a yield can also RECEIVE a value
        print("got", received)

e = echo()
next(e)          # "prime" it: run to the first yield
e.send("hi")     # got hi
e.close()        # raises GeneratorExit inside; generator ends
```

INTERVIEW: Interview: "why use a generator?"

"Memory efficiency and laziness: process huge or infinite streams one item at a time, build pipelines, and start producing results immediately. The cost is that it is one-shot and has no indexing or length."

4. Exception Handling

4.1 What an exception is

An **exception** is an object that Python raises when something goes wrong at run time. Raising it **stops normal execution** and unwinds the call stack looking for a matching **except**. If nobody handles it, the program stops and prints a **traceback**. (A **SyntaxError** is different: it is found before the program runs.)

DIAGRAM: exception propagation

```
main() -> load() -> parse() -> int("abc")      raises ValueError
                                   |
parse(): no handler  <-----+   unwinds (its remaining code is skipped)
load():  no handler  <-----+   unwinds
main():  try / except ValueError: HANDLED -> program continues here

If NO frame handles it -> traceback printed, program ends.
```

4.2 try / except / else / finally

```
try:
    risky()                # 1. code that might fail
except SomeError as e:
    handle(e)              # 2. runs ONLY if SomeError was raised in try
else:
    on_success()           # 3. runs ONLY if try finished with NO exception
finally:
    cleanup()              # 4. ALWAYS runs (error, success, return, break, continue)
```

Situation	Blocks that run, in order
No exception	try -> else -> finally
Exception, a matching except exists	try (up to the error) -> except -> finally
Exception, no matching except	try (up to the error) -> finally -> exception keeps propagating
Exception raised inside except	finally runs, then the new exception propagates
return inside try	return value is computed, finally runs, then the function returns

CONCEPT: Why have else?

Code in **else** runs only when **try** succeeded, and exceptions raised in **else** are **not** caught by the preceding **except**. That keeps the **try** body tiny and precise: only the line that can fail lives there.

4.3 Handling several exceptions

```
def safe_div(a, b):
    try:
        result = int(a) / int(b)
    except ValueError:
        print("not an integer")
    except ZeroDivisionError:
        print("cannot divide by zero")
    except (TypeError, KeyError) as e:      # several types in one tuple
        print("type/key problem:", e)
    except Exception as e:                  # broad catch-all LAST
        print("something else:", type(e).__name__)
    else:
        print("ok:", result)
    finally:
        print("done")

safe_div("10", "2")    # ok: 5.0 / done
safe_div("x", "2")     # not an integer / done
safe_div("1", "0")     # cannot divide by zero / done
```

- Python tries the **except** clauses **top to bottom** and uses the first match.
- Put **specific** exceptions before **general** ones. A general clause above a specific one makes the specific one unreachable.
- **except Exception** catches almost everything you should catch; it does **not** catch **KeyboardInterrupt** or **SystemExit** (they derive from **BaseException**).

COMMON TRAP: The exception variable disappears after the block

```
try:
    1 / 0
except ZeroDivisionError as e:
    pass
# print(e)      # NameError: e was deleted when the except block ended
err = None
try:
    1 / 0
except ZeroDivisionError as e:
    err = e      # save it in another name if you need it later
```

4.4 raise, re-raise, chaining

```
def set_age(age):
    if age < 0:
        raise ValueError("age cannot be negative")    # raise an instance
    return age

try:
    set_age(-1)
except ValueError as e:
    print("caught:", e)          # caught: age cannot be negative

try:
    try:
        int("abc")
    except ValueError:
        print("logging the problem...")
        raise                # bare raise: re-raise the SAME exception
except ValueError:
    print("re-raised and caught again")

try:
    try:
        {}["k"]
    except KeyError as e:
        raise RuntimeError("config broken") from e    # chaining: keeps the cause
except RuntimeError as e:
    print(type(e.__cause__).__name__)                  # KeyError
```

4.5 finally details and the return trap

```
def f():
    try:
        return "try"
    finally:
        print("finally runs first")

print(f())          # finally runs first / try

def g():
    try:
        return 1
    finally:
        return 2      # overrides the earlier return (and would swallow an exception)

print(g())          # 2
```

COMMON TRAP: return, break or continue inside finally

A **return** in **finally** silently **replaces** the pending return value and **discards** any pending exception. Avoid it; linters flag it.

4.6 Custom exceptions

Create your own exception by subclassing **Exception** (never **BaseException**). Custom exceptions give your code a vocabulary, let callers catch exactly what they want, and can carry extra data.

```
class AppError(Exception):
    """Base class for all errors in this application."""

class InsufficientFundsError(AppError):
    def __init__(self, balance, needed):
        super().__init__(f"balance {balance} is less than needed {needed}")
        self.balance = balance
        self.needed = needed

class Account:
    def __init__(self, balance):
        self.balance = balance
    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount

try:
    Account(50).withdraw(80)
except AppError as e:
    # catches the whole family
    print(e, "| short by", e.needed - e.balance)
# balance 50 is less than needed 80 | short by 30
```

4.7 Common built-in exceptions

Exception	Typical cause	Example
ZeroDivisionError	Division or modulo by zero	<code>1 / 0</code>
ValueError	Right type, unacceptable value	<code>int("abc")</code>
TypeError	Operation on the wrong type / wrong call signature	<code>"a" + 1, len(5)</code>
IndexError	Sequence index out of range	<code>[1, 2][5]</code>
KeyError	Missing dictionary key	<code>{x}[x]</code>
AttributeError	Object has no such attribute	<code>None.append(1)</code>
NameError	Name not defined	<code>print(zzz)</code>
UnboundLocalError	Local variable used before assignment (subclass of NameError)	see section 1.9
ImportError / ModuleNotFoundError	Import failed	<code>import nomodule</code>
FileNotFoundError	File missing (subclass of OSError)	<code>open("no.txt")</code>
StopIteration	<code>next()</code> on an exhausted iterator	<code>next(iter([]))</code>
RecursionError	Recursion too deep (default limit about 1000)	function without a base case
AssertionError	<code>assert</code> condition false	<code>assert 1 == 2</code>
NotImplementedError	Placeholder for methods subclasses must define	<code>raise NotImplementedError</code>
RuntimeError	Generic runtime problem	dict changed size during iteration
KeyboardInterrupt	User pressed Ctrl+C (BaseException , not Exception)	
OverflowError / MemoryError	Number too large for a float op / out of memory	<code>2.0 ** 10000</code>

DIAGRAM: built-in exception hierarchy (simplified)

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception          <- what you normally catch
    +-- ArithmeticError --> ZeroDivisionError, OverflowError
    +-- LookupError      --> IndexError, KeyError
    +-- ValueError
    +-- TypeError
    +-- NameError        --> UnboundLocalError
    +-- AttributeError
    +-- OSError          --> FileNotFoundError, PermissionError
    +-- ImportError      --> ModuleNotFoundError
    +-- RuntimeError     --> RecursionError, NotImplementedError
    +-- StopIteration
    +-- AssertionError
```

Because of the hierarchy, **except** **LookupError** catches both **IndexError** and **KeyError**, and **except** **ArithmeticError** catches **ZeroDivisionError**.

4.8 Best practices

Do	Avoid
Catch specific exceptions	Bare except : (hides KeyboardInterrupt , typos, everything)
Keep the try body as small as possible	Wrapping 50 lines in one try
Use finally or with for cleanup (files, locks, connections)	Relying on code after the try to clean up
Re-raise (raise) or wrap (raise X from e) when you cannot handle it	except Exception: pass (silent failure)
Write helpful messages with the offending value	raise Exception("error")
Subclass Exception for custom errors; make a base class per package	Returning error codes/ None where an exception is clearer
Log errors (logging module) with context	Using assert to validate user input (asserts can be disabled with -O)

```
# LBYL: "Look Before You Leap"
if key in d:
    value = d[key]
else:
    value = 0
# Both fine. Python style often prefers EAFP; d.get(key, 0) is shortest here.

# EAFP: "Easier to Ask Forgiveness than Permission"
try:
    value = d[key]
except KeyError:
    value = 0

with open("data.txt") as f:      # context manager: f.close() guaranteed, even on error
    text = f.read()
```

INTERVIEW: Interview: order of execution puzzle

Always trace **try -> except/else -> finally**. Remember: **finally** runs **even when try** has a **return**, and a **return** inside **finally** wins.

5. Object-Oriented Programming

5.1 Class, object, self, __init__

A **class** is a blueprint; an **object** (instance) is one thing built from that blueprint. Each object has its own data (**instance variables**) and shares the class's methods and class variables.

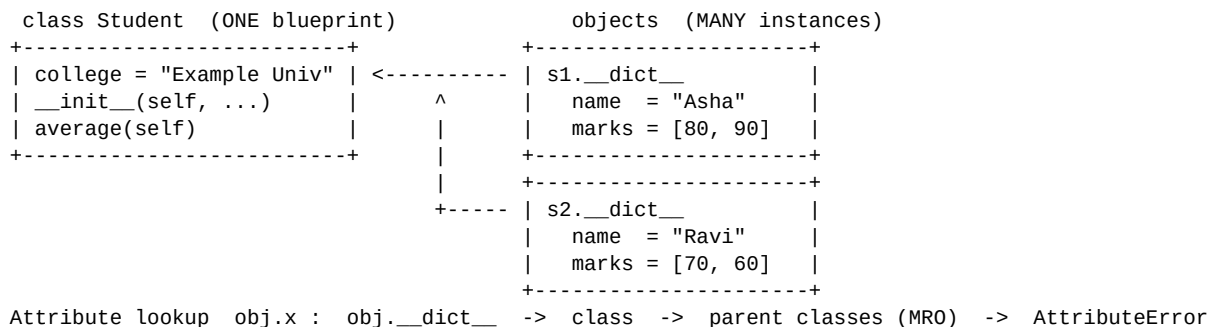
```
class Student:
    college = "Example University"      # class variable: shared by all instances

    def __init__(self, name, marks):    # initializer ("constructor")
        self.name = name                # instance variables: one copy per object
        self.marks = marks

    def average(self):                  # instance method
        return sum(self.marks) / len(self.marks)

s1 = Student("Asha", [80, 90])
s2 = Student("Ravi", [70, 60])
print(s1.average())                   # 85.0
print(Student.average(s1))             # 85.0    <- identical: s1.average() is sugar for this
```

DIAGRAM: class vs instances



- ****self**** is not a keyword; it is simply the conventional name of the first parameter. Python passes the object there automatically. It tells a method *which* object it is working on.
- ****__init__**** initializes an object that already exists. It must return **None**. It is often called the constructor, but strictly the object is created earlier by ****__new__(cls, ...)****, which you rarely override (singletons, immutable subclasses).
- Instance variables are usually created in **__init__**, but Python allows adding them anywhere (**obj.new_attr = 1**).

5.2 Instance variables vs class variables

	Instance variable	Class variable
Defined	Inside methods via self.x = ...	In the class body, outside methods
Stored in	The instance (obj.__dict__)	The class (cls.__dict__)
Shared?	No, each object has its own	Yes , all instances see the same one
Access	obj.x	cls.x (or obj.x if no instance variable shadows it)

```

class Counter:
    count = 0                # class variable
    def __init__(self):
        Counter.count += 1   # modify through the CLASS name

a = Counter()
b = Counter()
print(Counter.count, a.count)    # 2 2

a.count = 99                  # creates an INSTANCE attribute that shadows
print(a.count, b.count, Counter.count)  # 99 2 2

```

COMMON TRAP: Mutable class variables are shared by everyone

```

class Team:
    members = []             # ONE list for the whole class
    def add(self, m):
        self.members.append(m) # mutates the shared list

t1, t2 = Team(), Team()
t1.add("Asha")
print(t2.members)           # ['Asha'] surprise!
# Fix: create it per object in __init__: self.members = []

```

Also beware **self.count += 1** when **count** is only a class variable: it reads the class value, then creates an **instance** attribute holding the result.

5.3 Instance, class and static methods

Kind	Decorator	First parameter	Can touch instance data?	Typical use
Instance method	none	self (the object)	Yes	Normal behaviour of an object
Class method	@classmethod	cls (the class)	No; can touch class data	Alternative constructors, factory methods
Static method	@staticmethod	none	No	Utility function that logically belongs to the class

```

class Temp:
    unit = "C"
    def __init__(self, deg):
        self.deg = deg

    def show(self):                # instance method
        return f"{self.deg}{self.unit}"

    @classmethod
    def from_f(cls, f):           # alternative constructor
        return cls((f - 32) * 5 / 9) # cls, not Temp: works for subclasses too

    @staticmethod
    def is_valid(deg):             # no self, no cls
        return deg >= -273.15

t = Temp.from_f(212)
print(t.show())                  # 100.0C
print(Temp.is_valid(-300))       # False

```

5.4 Encapsulation

Encapsulation = bundling data with the methods that operate on it, and controlling access to the internals. Python has **no truly private** attributes; it relies on naming conventions and properties.

Naming	Meaning	Enforced?
<code>name</code>	Public	-
<code>_name</code>	"Internal, please do not touch" (protected by convention)	No
<code>__name</code>	Name-mangled to <code>_ClassName__name</code> ; avoids accidental clashes in subclasses	Only by renaming, still reachable

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance          # internal by convention
        self.__pin = 1234                # mangled to _BankAccount__pin

    @property
    def balance(self):                    # getter: acc.balance
        return self._balance

    @balance.setter
    def balance(self, value):             # setter: acc.balance = ...
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value

acc = BankAccount(100)
acc.balance = 150                        # runs the setter (validation)
print(acc.balance)                       # 150 runs the getter
print(acc._BankAccount__pin)             # 1234 mangled, not truly hidden
```

TIP: Why @property?

It lets you start with a plain public attribute and later add validation or computed values **without changing how callers use it** (`acc.balance` stays `acc.balance`). Do not write Java-style `get_x()` / `set_x()` methods unless needed.

5.5 Abstraction

Abstraction = exposing what an object does and hiding how. In Python, abstract base classes (**abc** module) force subclasses to implement required methods.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2          # must implement, or Circle stays abstract

# Shape()          # TypeError: Can't instantiate abstract class Shape
print(Circle(2).area())    # 12.56636
```

5.6 Inheritance, overriding, super()

Inheritance lets a child class reuse and extend a parent class (an **is-a** relationship). A child can **override** a method by defining one with the same name. **super()** gives access to the next class in the method resolution order, most simply the parent.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)          # reuse the parent's initializer
        self.breed = breed
    def speak(self):                    # overriding
        return f"{self.name} barks"

d = Dog("Rex", "Lab")
print(d.speak())                       # Rex barks
print(isinstance(d, Animal), isinstance(Dog, Animal)) # True True
print(type(d) == Animal, type(d) == Dog)          # False True
```

- Forms: single, multilevel (A > B > C), hierarchical (one parent, many children), multiple (several parents), hybrid.
- **isinstance(obj, cls)** is True for subclasses too; **type(obj) == cls** is exact. Prefer **isinstance**.
- If the child defines **__init__** and forgets **super().__init__()**, the parent's initialization never runs.

5.7 Polymorphism and duck typing

Polymorphism = the same operation works on different types, each behaving in its own way. In Python it appears as method **overriding**, as **duck typing** ("if it walks like a duck and quacks like a duck..."), and as **operator/built-in overloading** (**len()**, **+**, **print()** work on many types).

```
class Duck:
    def speak(self): return "quack"
class Robot:
    def speak(self): return "beep"          # no common parent needed

for thing in [Duck(), Robot()]:
    print(thing.speak())                   # quack / beep
```

5.8 Method overloading in Python

Python has **no compile-time method overloading**. If you define two methods with the same name in a class, the **later definition silently replaces the earlier one**. You get the effect of overloading with:

- **Default arguments** and ***args / **kwargs**.
- **Type checks** with **isinstance** inside one method.
- **functools.singledispatch** (dispatch on the type of the first argument).
- **Class methods** as named alternative constructors (**from_f**, **from_string**).
- **Operator overloading** via dunder methods (**__add__**, **__eq__**, ...) is fully supported.

```

class Calc:
    def add(self, a, b=0, c=0):          # default args emulate 1-, 2-, 3-arg versions
        return a + b + c

c = Calc()
print(c.add(1), c.add(1, 2), c.add(1, 2, 3))    # 1 3 6

from functools import singledispatch

@singledispatch
def show(x):
    return "other"

@show.register(int)
def _(x):
    return "int"

@show.register(str)
def _(x):
    return "str"

print(show(1), show("a"), show(2.5))          # int str other

```

5.9 Composition

Composition = building a class by holding other objects (a **has-a** relationship). Prefer it when the relationship is not truly "is-a": it is more flexible and avoids deep, fragile inheritance trees.

Relationship	Sentence test	Tool	Example
is-a	"A Dog is an Animal"	Inheritance	class Dog(Animal)
has-a	"A Car has an Engine"	Composition	self.engine = Engine()

```

class Engine:
    def start(self):
        return "engine on"

class Car:
    def __init__(self):
        self.engine = Engine()          # Car HAS-A Engine
    def start(self):
        return self.engine.start() + ", car moving"

print(Car().start())                   # engine on, car moving

```

5.10 Multiple inheritance and MRO

A class may list several parents: **class D(B, C)**. When a method exists in more than one ancestor, Python follows the **Method Resolution Order (MRO)**, computed by the **C3 linearization** algorithm. See it with **D.__mro__** or **D.mro()**.

DIAGRAM: the diamond

```

    A
  /  \
 B    C
  \  /
    D

```

```

class A: ...
class B(A): ...
class C(A): ...
class D(B, C): ...

D.__mro__ -> D, B, C, A, object

```

```

class A:
    def f(self): print("A")
class B(A):
    def f(self):
        print("B"); super().f()
class C(A):
    def f(self):
        print("C"); super().f()
class D(B, C):
    def f(self):
        print("D"); super().f()

D().f()
print([k.__name__ for k in D.__mro__])
# D B C A
# ['D', 'B', 'C', 'A', 'object']

```

MENTAL MODEL: What super () really means

super () does **not** mean "my parent". It means **"the next class after me in the MRO of the object's actual type"**. In the diamond above, **B's super () . f ()** calls **C . f**, not **A . f**, because the MRO of a **D** object is D, B, C, A. This is **cooperative inheritance**: every class in the chain calls **super ()** so each runs exactly once.

CONCEPT: C3 by hand (how to compute an MRO in an exam)

$L[X] = X + \text{merge}(L[\text{parent1}], L[\text{parent2}], \dots, [\text{parent1}, \text{parent2}, \dots])$. Repeatedly take the first head that does **not** appear in the tail of any other list.

- $L[A] = [A, \text{object}]$, $L[B] = [B, A, \text{object}]$, $L[C] = [C, A, \text{object}]$
- $L[D] = D + \text{merge}([B, A, o], [C, A, o], [B, C])$
- take B (in no tail) -> merge($[A, o]$, $[C, A, o]$, $[C]$). A is in the tail of $[C, A, o]$, skip it; take C
- merge($[A, o]$, $[A, o]$, $[]$) -> A, then object. Result: **D, B, C, A, object**

COMMON TRAP: Inconsistent MRO

```

class A: pass
class B(A): pass
class X(A, B): pass      # TypeError: Cannot create a consistent method resolution order
# A must come before its subclass B (children first), but the base list says A first.
class Y(B, A): pass      # correct: subclass before base

```

- A **mixin** is a small class meant to be combined with others to add one capability (**LoggingMixin**). It is multiple inheritance used well.
- Old Python 2 "classic" classes used depth-first search; Python 3 always uses C3.

5.11 Dunder (magic) methods

Dunder = "double underscore" methods such as `__init__`. Python calls them **implicitly** when you use syntax or built-ins. You never call `obj.__len__()` yourself; you write `len(obj)`. Defining them makes your class behave like a built-in type.

Method	Triggered by	Purpose / note
<code>__init__(self, ...)</code>	<code>cls(...)</code>	Initialize a new instance
<code>__str__(self)</code>	<code>str(obj)</code> , <code>print(obj)</code> , f-strings	Readable text for users
<code>__repr__(self)</code>	<code>repr(obj)</code> , REPL, items inside containers	Unambiguous text for developers ; ideally looks like the constructor call
<code>__len__(self)</code>	<code>len(obj)</code>	Also used by <code>bool(obj)</code> if <code>__bool__</code> is missing
<code>__eq__(self, other)</code>	<code>a == b</code>	Value equality. Defining it without <code>__hash__</code> makes instances unhashable
<code>__hash__(self)</code>	<code>hash(obj)</code> , dict keys, set members	Equal objects must have equal hashes
<code>__add__(self, other)</code>	<code>a + b</code>	Return a new object. Return NotImplemented for unsupported types
<code>__radd__</code> , <code>__iadd__</code>	<code>5 + obj</code> , <code>obj += x</code>	Reflected and in-place versions

Method	Triggered by	Purpose / note
<code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , <code>__ge__</code>	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , and <code>sorted()</code>	<code>functools.total_ordering</code> fills in the rest
<code>__getitem__(self, i)</code>	<code>obj[i]</code> , also legacy iteration	Indexing/slicing
<code>__contains__</code> , <code>__iter__</code> , <code>__next__</code>	<code>x in obj</code> , for loops	Membership and iteration protocol
<code>__bool__</code>	<code>bool(obj)</code> , <code>if obj:</code>	Truthiness
<code>__call__</code>	<code>obj(...)</code>	Make instances callable
<code>__enter__</code> , <code>__exit__</code>	<code>with obj:</code>	Context manager protocol

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"      # developer view

    def __str__(self):
        return f"({self.x}, {self.y})"          # user view

    def __len__(self):
        return 2

    def __eq__(self, other):
        return isinstance(other, Vector) and (self.x, self.y) == (other.x, other.y)

    def __hash__(self):
        return hash((self.x, self.y))           # keep hashability since we defined __eq__

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __getitem__(self, i):
        return (self.x, self.y)[i]

v = Vector(1, 2) + Vector(3, 4)
print(v)                # (4, 6)                uses __str__
print([v])               # [Vector(4, 6)]        containers use __repr__
print(repr(v))           # Vector(4, 6)
print(len(v), v == Vector(4, 6), v[0])          # 2 True 4
```

CONCEPT: `__str__` vs `__repr__` fallback

`print(obj)` uses `__str__`; if it is missing Python falls back to `__repr__`. The reverse is **not** true: with only `__str__`, `repr(obj)` (and lists of objects) still show `<Cls object at 0x...>`. ****Rule of thumb: always define `__repr__`; add `__str__` only if you want a friendlier display.****

INTERVIEW: Interview: equality and hashing

"If I define `__eq__` and nothing else, Python sets `__hash__ = None`, so my objects can no longer be dict keys or set members. If objects are mutable, being hashable on mutable fields is dangerous, so hash only fields that never change."

6. Diagram Gallery

Ten pictures worth memorising. If you can redraw these from scratch, you understand the chapter.

6.1 Mutation vs rebinding

DIAGRAM

BEFORE

a --> [1,2,3]
b --> [1,2,3] (same)

MUTATION: a.append(4)

a --> [1,2,3,4]
b --> [1,2,3,4] (same obj)

REBINDING: a = [7]

a --> [7]
b --> [1,2,3] (old object)

Mutation changes THE OBJECT (every name sees it).
Rebinding moves ONE NAME (other names unaffected).

6.2 The iterator protocol as a conversation

DIAGRAM

```
for x in data:                data (iterable)          it (iterator)
|                             |                         |
|----- iter(data) ----->|                         |
|<----- returns it -----|                         |
|----- next(it) ----->|                         |
|<----- value 1 ----->|                         |
|----- next(it) ----->|                         |
|<----- value 2 ----->|                         |
|----- next(it) ----->|                         |
|<----- raises StopIteration -----|             |
v                                                     |
loop ends quietly (else-block of the loop runs if there was no break)
```

6.3 List vs generator in memory

DIAGRAM

squares_list = [x*x for x in range(5)] squares_gen = (x*x for x in range(5))

+-----+ 0 1 4 9 16 +-----+ index [0]..[4], len() = 5	all stored (O(n) memory)	+-----+ generator object state: paused at x = 2 code : x*x for x in range(5) +-----+ one value at a time (O(1) memory)
---	-----------------------------	---

6.4 try / except / else / finally flow

DIAGRAM

```

+-----+
|  try:  |
+-----+
no error | error raised
+-----+
v       v
+-----+           matching except?
|  else:  |         yes /      \ no
+-----+           v         v
|           |         +-----+ (exception keeps
|           |         | except: | propagating)
|           |         +-----+
+-----+           |
v                   |
+-----+           |
| finally: |         ALWAYS runs
+-----+
```

6.5 Attribute lookup

DIAGRAM

```
obj.speak          (obj is an instance of Dog; Dog(Animal))

1. obj.__dict__      instance attributes      found? -> use it
2. Dog.__dict__      the class                found? -> use it
3. Animal.__dict__   next class in the MRO    found? -> use it
4. object.__dict__   last class in the MRO    found? -> use it
5. nothing found     -> AttributeError

(properties/descriptors and __getattr__ add extra rules, but this is the core.)
```

6.6 Bound methods

DIAGRAM

```
s1.average()      ==      Student.average(s1)
  ^                      ^
  |                      +-- s1 is passed in automatically as self
+-- Python finds average on the class, BINDS it to s1 (a "bound method")

@classmethod -> first argument is the class (cls), Temp.from_f(212)
@staticmethod -> nothing is passed automatically, Temp.is_valid(5)
```

6.7 C3 merge, step by step

DIAGRAM

```
class D(B, C)      L[B] = [B, A, o]      L[C] = [C, A, o]      bases = [B, C]

merge( [B,A,o], [C,A,o], [B,C] )
  take B (not in any tail)      -> result: D B
merge( [A,o], [C,A,o], [C] )
  A? A is in tail of [C,A,o] SKIP
  take C      -> result: D B C
merge( [A,o], [A,o], [] )
  take A, then o      -> result: D B C A o
```

6.8 Closures and cells

DIAGRAM

```
def make_counter():          counter = make_counter()
  c = 0          <-----+
  def inc():      |          counter --> function inc
    nonlocal c    |
    c += 1        +----- cell "c" <-----+ (kept alive by the closure)
    return c
  return inc          each call: c = c + 1 inside the SAME cell
```

6.9 Everything is an object

DIAGRAM

```
5 ---- is an instance of ----> int ---- is an instance of ----> type
"hi" ---- is an instance of ----> str ---- is an instance of ----> type
Dog()---- is an instance of ----> Dog ---- is an instance of ----> type

Classes are objects too. Every class inherits from object. type(type) is type.
```

6.10 Exception propagation with finally

DIAGRAM

```
def a(): try: b() finally: print("a-cleanup")
def b(): try: c() finally: print("b-cleanup")
def c(): raise ValueError

a() -> b() -> c() raises
      c unwinds (no handler)
      b: finally runs -> prints "b-cleanup"
      a: finally runs -> prints "a-cleanup"
      caller: except ValueError: handled here (or traceback if not)
```

7. Terminology Cheat Sheet

Term	Meaning
Name / variable	A label bound to an object; not a memory box
Object	A value with identity, type and (possibly changeable) contents
Reference	The link between a name (or container slot) and an object
Binding / rebinding	Making / changing what a name points to (x = ...)
Identity	Who the object is: id(obj) , compared with is
Aliasing	Multiple names referring to one object
Mutable / immutable	Object can / cannot change in place
Hashable	Has a stable hash and __eq__ ; usable as dict key / set member
Shallow copy	New outer container, shared inner objects
Deep copy	Recursive duplicate; nothing shared; handles cycles via memo
Pass by object reference	Parameter is a local name bound to the caller's object (call by sharing)
Default argument	Value evaluated once at def time and stored on the function
Reference counting	CPython frees an object when its count reaches 0
Garbage collector	Frees cyclic garbage that reference counting cannot
LEGB	Local, Enclosing, Global, Built-in: the name lookup order
Closure	Inner function that remembers variables of its enclosing function
Late binding	Closures read variables when called, not when defined
global / nonlocal	Allow rebinding a module-level / enclosing-function name
Iterable	Can be looped over; has __iter__
Iterator	Has __next__ ; remembers position; one-shot
Iterator protocol	__iter__ + __next__ + StopIteration
StopIteration	Signal that an iterator has no more items
Exhausted	An iterator that has already raised StopIteration
Sentinel	Special value that marks the end (iter(f, sentinel))
Generator function	Function containing yield ; returns a generator object
Generator expression	(expr for x in it) : lazy one-shot iterator
yield / yield from	Pause and produce a value / delegate to a sub-iterable
Lazy evaluation	Compute values only when needed
Coroutine (generator style)	Generator that also receives values with send()
Exception	Object raised on error; unwinds the stack until handled
Traceback	Report of the call stack at the moment of an unhandled exception
try / except / else / finally	Guarded code / handler / success-only / always-run cleanup
Re-raise	Bare raise inside except : pass the same exception on
Exception chaining	raise X from e : keeps e as __cause__
EAFP / LBYL	Try then handle failure / check before acting
Context manager	Object for with : __enter__ and __exit__ guarantee cleanup
Class	Blueprint defining attributes and methods
Instance / object	A concrete thing created from a class
self	The instance passed as first argument to instance methods
cls	The class passed as first argument to class methods
Constructor / initializer	__init__ initializes; __new__ creates
Instance / class variable	Per-object data / data shared by the class

Term	Meaning
Instance / class / static method	Takes self / cls / neither
Encapsulation	Bundling data with methods and controlling access
Name mangling	<code>__x</code> inside class C becomes <code>_C__x</code>
Property	Method exposed with attribute syntax via @property
Abstraction / ABC	Hide details; abc.ABC + @abstractmethod force implementation
Inheritance	Child class reuses and extends a parent (is-a)
Overriding	Child redefines an inherited method
Overloading	Same name, several signatures: emulated in Python; operator overloading via dunder
Polymorphism / duck typing	Same call works on many types; only needed methods matter
Composition	Object holds other objects (has-a)
super()	Delegates to the next class in the MRO
MRO	Method Resolution Order, computed by C3 linearization
Diamond problem	Two paths to one ancestor via multiple inheritance
Mixin	Small class adding one capability, meant to be combined
Dunder / magic method	Special <code>__name__</code> method called implicitly by Python
<code>__repr__</code> / <code>__str__</code>	Developer text / user text representation
NotImplemented	Return value telling Python to try the other operand's method

8. Question Bank

Attempt each section **without looking at the answers**. Then check against Section 9. Aim for at least 80% on each list before moving on. Labels: **C** = conceptual, **O** = output, **I** = interview.

8A. 40 Conceptual Questions

- C1. What is a variable in Python, really?
- C2. What are the three properties of every object, and what does `id()` return?
- C3. Difference between `==` and `is`?
- C4. What is aliasing and how is it created?
- C5. Define mutable and immutable. Does immutable mean a variable cannot change?
- C6. Shallow copy vs deep copy?
- C7. Name four ways to shallow-copy a list. Which common assignment is not a copy?
- C8. Why does `[[0]*3]*3` misbehave?
- C9. How are arguments passed to functions in Python?
- C10. Why is a mutable default argument dangerous and what is the fix?
- C11. Iterable vs iterator?
- C12. What do `iter()`, `next()` and `StopIteration` do?
- C13. How does a `for` loop work internally?
- C14. Why can an iterator be consumed only once? How do you iterate again?
- C15. What must a class define to be (a) an iterator, (b) an iterable?
- C16. What is a generator function and what does `yield` do?
- C17. Explain lazy evaluation and the memory benefit of generators.
- C18. Generator expression vs list comprehension?
- C19. `yield` vs `return`?
- C20. When should you prefer a list to a generator?
- C21. What happens when an exception is not caught?
- C22. Explain the roles of `try`, `except`, `else`, `finally`.
- C23. Multiple `except` clauses: how are they chosen, and what is the tuple form?
- C24. What do `raise`, bare `raise`, and `raise X from e` do?
- C25. Why create custom exceptions and how?
- C26. Why is a bare `except:` bad? What should you catch?
- C27. EAFP vs LBYL?
- C28. Class vs object?
- C29. What is `self`?
- C30. `__init__` vs `__new__`?
- C31. Instance variable vs class variable?
- C32. Instance vs class vs static method?
- C33. How does Python do encapsulation?
- C34. What is abstraction and how do you enforce it?
- C35. What is inheritance and what does `super()` do?
- C36. Polymorphism, duck typing, method overriding?
- C37. Does Python support method overloading?
- C38. Composition vs inheritance?
- C39. What is the MRO and what problem does multiple inheritance create?
- C40. What are dunder methods and how are they used?

8B. 40 Output Questions (predict the output)

Write down the exact output, or **Raises SomeError**, before checking. Assume each snippet runs as a fresh script.

O1

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

O2

```
a = [1, 2, 3]
b = a[:]
b.append(4)
print(a, b)
```

O3

```
a = [[1], [2]]
b = a.copy()
b[0].append(9)
print(a)
```

O4

```
import copy
a = [[1], [2]]
b = copy.deepcopy(a)
b[0].append(9)
print(a, b)
```

O5

```
m = [[0] * 3] * 3
m[0][0] = 1
print(m)
```

O6

```
m = [[0] * 3 for _ in range(3)]
m[0][0] = 1
print(m)
```

O7

```
def f(lst):
    lst.append(1)
x = []
f(x)
f(x)
print(x)
```

O8

```
def f(lst):
    lst = lst + [1]
x = []
f(x)
print(x)
```

O9

```
def f(lst):
    lst += [1]
x = []
f(x)
print(x)
```

O10

```
def f(n, acc=[]):
    acc.append(n)
    return acc
print(f(1))
print(f(2))
print(f(3, []))
```

O11

```
x = 10
def f():
    x = 20
f()
print(x)
```

O12

```
x = 10
def f():
    print(x)
    x = 5
f()
```

O13

```
def outer():
    c = 0
    def inner():
        nonlocal c
        c += 1
        return c
    return inner
f = outer()
print(f(), f(), f())
```

O14

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

O15

```
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])
```

O16

```
t = ([1, 2],)
t[0].append(3)
print(t)
```

O17

```
t = ([1, 2],)
try:
    t[0] += [3]
except TypeError:
    print("error")
print(t)
```

O18

```
s = "abc"
t = s
s += "d"
print(s, t)
```

O19

```
a = [1, 2, 3]
it = iter(a)
print(next(it))
print(list(it))
print(list(it))
```

O20

```
it = iter([1, 2])
print(next(it), next(it), next(it, "end"))
```

O21

```
r = range(3)
print(list(r), list(r))
```

O22

```
m = map(str, [1, 2])
print(list(m), list(m))
```

O23

```
def g():
    print("A")
    yield 1
    print("B")
    yield 2
x = g()
print("start")
print(next(x))
print(next(x))
```

O24

```
def g():
    yield 1
    return 99
print(list(g()))
```

O25

```
g = (x * x for x in range(3))
print(sum(g), sum(g))
```

O26

```
x = 1
g = (x * i for i in range(3))
x = 10
print(list(g))
```

O27

```
def g():
    yield from [1, 2]
    yield 3
print(list(g()))
```

O28

```
def f():
    try:
        return "try"
    finally:
        print("finally")
print(f())
```

O29

```
def f():
    try:
        return 1
    finally:
        return 2
print(f())
```

O30

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

O31

```
try:
    print("ok")
except Exception:
    print("A")
else:
    print("B")
finally:
    print("C")
```

O32

```
class A:
    n = 0
    def __init__(self):
        self.n += 1
a = A()
b = A()
print(A.n, a.n, b.n)
```

O33

```
class A:
    items = []
    def add(self, x):
        self.items.append(x)
a = A()
b = A()
a.add(1)
print(b.items)
```

O34

```
class A:
    def show(self):
        print("A")
class B(A):
    def show(self):
        super().show()
        print("B")
B().show()
```

O35

```
class A:
    def f(self):
        print("A")
class B(A):
    def f(self):
        print("B")
        super().f()
class C(A):
    def f(self):
        print("C")
        super().f()
class D(B, C):
    def f(self):
        print("D")
        super().f()
D().f()
print([k.__name__ for k in D.__mro__])
```

O36

```
class P:
    def __init__(self, x):
        self.x = x
    def __repr__(self):
        return f"P({self.x})"
    def __str__(self):
        return "point"
p = P(1)
print(p)
print([p])
print(repr(p))
```

O37

```
class V:
    def __init__(self, a):
        self.a = a
    def __add__(self, o):
        return V(self.a + o.a)
    def __eq__(self, o):
        return self.a == o.a
print((V(1) + V(2)) == V(3))
```

O38

```
class S:
    def __len__(self):
        return 0
print(bool(S()), len(S()))
```

O39

```
class A:
    def __init__(self):
        self.__x = 1
a = A()
print(hasattr(a, "__x"), hasattr(a, "_A__x"))
```

O40

```
class A:
    def f(self, x):
        return "one"
    def f(self, x, y):
        return "two"
print(A().f(1, 2))
```

8C. 40 Interview Questions

- I1. Why do `a = b = []` and `a = []; b = []` behave differently?
- I2. Create a 3x4 grid of zeros safely and explain why `[[0]*4]*3` is wrong.
- I3. A tuple is immutable, yet `t[0].append(1)` works when `t = ([],)`. Explain.
- I4. What happens with `t = ([1, 2],); t[0] += [3]`?
- I5. Write your own deep copy for nested lists/dicts. What does `copy.deepcopy` add?
- I6. Why is `x is None` preferred over `x == None`?
- I7. What is small-integer caching and why should you not rely on it?
- I8. `b = a; a += [4]` versus `b = a; a = a + [4]`: difference?
- I9. How does Python manage memory?
- I10. Explain the mutable default argument bug, when the default is evaluated, and the fix.
- I11. Why does `[lambda: i for i in range(3)]` return `[2, 2, 2]` and how do you fix it?
- I12. `global` vs `nonlocal`, and why does `UnboundLocalError` happen?
- I13. Why is removing items from a list while looping over it buggy, and what is the correct way?
- I14. Iterable vs iterator: is a file an iterator? Is `range`?
- I15. Implement an iterator class that yields Fibonacci numbers up to a limit.
- I16. Why does changing a dict during iteration raise `RuntimeError`, and how do you avoid it?
- I17. What does the two-argument form `iter(callable, sentinel)` do?
- I18. Generator vs list: trade-offs?
- I19. What exactly happens when you call a generator function?
- I20. Can a generator use `return`? What happens to the value?
- I21. What does `yield from` do?
- I22. Sum the squares of the first million integers without building a list.
- I23. A generator is exhausted. How can you traverse the data again?
- I24. What does `send()` do on a generator?

- I25. Trace: **try** has a **return**, **finally** prints. In what order do things happen?
- I26. Why put code in **else** instead of at the end of **try**?
- I27. Why does **except Exception** not catch Ctrl+C?
- I28. **raise** vs **raise X from e**; what happens with a bare **raise** outside an **except**?
- I29. Design a small custom exception hierarchy for a banking app.
- I30. Difference between **__init__** and **__new__**? Implement a singleton.
- I31. Class variables vs instance variables: what is the **self.count += 1** trap?
- I32. When would you use **@classmethod** versus **@staticmethod**?
- I33. Is Python encapsulation "real"? Explain name mangling.
- I34. What is **@property** and why use it?
- I35. How would you achieve method overloading in Python?
- I36. Explain the MRO for a diamond, and what **super()** really does.
- I37. Why should every class in a multiple-inheritance chain call **super().__init__()**?
- I38. What happens to **__hash__** when you define **__eq__**? Why does it matter?
- I39. **__str__** vs **__repr__**: which is used where?
- I40. Composition over inheritance: what does it mean? **isinstance** vs **type()**?

9. Complete Solutions

9A. Conceptual Answers

- C1. What is a variable in Python, really?** A name bound to an object (a reference). Assignment binds or rebinds the name; it never copies the object. Unlike C++, the type belongs to the object, not to the name.
- C2. What are the three properties of every object, and what does `id()` return?** Identity, type, value. `id()` returns the identity: a unique integer that stays constant for the object's lifetime (in CPython it is the memory address).
- C3. Difference between `==` and `is`?** `==` compares values (calls `__eq__`). `is` compares identity (same object). Use `is` only for `None`, `True`, `False`.
- C4. What is aliasing and how is it created?** Two or more names referring to the same object. Created by `b = a`, by passing an object to a function, by putting the same object in several containers, by `x = y = []`, and by `[obj] * n`.
- C5. Define mutable and immutable. Does immutable mean a variable cannot change?** Mutable objects can change in place (`list`, `dict`, `set`); immutable objects cannot (`int`, `str`, `tuple`, `frozenset`). The object is fixed, but the name can be rebound to a different object, so the variable can still "change".
- C6. Shallow copy vs deep copy?** A shallow copy creates a new outer container but shares the inner objects. A deep copy recursively duplicates everything, so nothing is shared.
- C7. Name four ways to shallow-copy a list. Which common assignment is not a copy?** `a[:]`, `a.copy()`, `list(a)`, `copy.copy(a)`. `b = a` is not a copy; it is an alias.
- C8. Why does `[[0]*3]*3` misbehave?** `*` repeats the reference to the inner list, so all three rows are the same object. Use `[[0]*3 for _ in range(3)]`, which builds a new list per row.
- C9. How are arguments passed to functions in Python?** By object reference (call by sharing): the parameter is a new local name bound to the caller's object. Mutation is visible to the caller; rebinding is not.
- C10. Why is a mutable default argument dangerous and what is the fix?** Defaults are evaluated once at definition time, so a default list/dict is shared across calls. Use `None` as the default and create the object inside the function.
- C11. Iterable vs iterator?** An iterable can be looped over and supplies an iterator via `__iter__`. An iterator has `__next__`, yields items one at a time, remembers position, and its `__iter__` returns itself. Lists are iterables; generators and files are iterators.
- C12. What do `iter()`, `next()` and `StopIteration` do?** `iter(x)` returns an iterator for `x`. `next(it)` returns the next item. `StopIteration` is raised by `next` when items run out; it signals the end of iteration. `next(it, default)` returns `default` instead of raising.
- C13. How does a for loop work internally?** It calls `iter()` on the iterable once, then repeatedly calls `next()` on the iterator, running the body each time, and stops silently when `StopIteration` is raised.
- C14. Why can an iterator be consumed only once? How do you iterate again?** It stores a single moving position and never rewinds. To iterate again, get a fresh iterator from the original iterable (`iter(lst)`), or recreate the generator.
- C15. What must a class define to be (a) an iterator, (b) an iterable?** (a) `__next__` and `__iter__` returning `self`. (b) `__iter__` that returns an iterator (often a new one each call, or a generator).
- C16. What is a generator function and what does `yield` do?** A function containing `yield`. Calling it returns a generator (an iterator) without running the body. `yield` hands out a value and pauses, preserving local state until the next `next()`.
- C17. Explain lazy evaluation and the memory benefit of generators.** Values are computed only when requested, so a generator holds just its current state (O(1) memory) rather than the full sequence (O(n)). It can even represent infinite sequences.
- C18. Generator expression vs list comprehension?** (`expr for x in it`) builds a lazy, one-shot generator; `[expr for x in it]` builds the whole list immediately. Generator: less memory, no indexing or `len()`.
- C19. `yield` vs `return`?** `return` ends the function and gives one value; local state is destroyed. `yield` gives a value and pauses, preserving state, so the function can produce many values over time.
- C20. When should you prefer a list to a generator?** When you need indexing, `len()`, slicing, multiple passes, sorting in place, or the data is small. Also when you need results computed once and reused.
- C21. What happens when an exception is not caught?** It propagates up the call stack, unwinding each frame (running `finally` blocks and `with` cleanups). If nothing handles it, the program terminates with a traceback.
- C22. Explain the roles of `try`, `except`, `else`, `finally`.** `try`: code that might fail. `except`: runs if a matching exception occurred. `else`: runs only if no exception occurred. `finally`: always runs, used for cleanup.

- C23. Multiple except clauses: how are they chosen, and what is the tuple form?** The first matching clause (top to bottom) runs, so put specific exceptions before general ones. **except (A, B) as e:** handles several types the same way.
- C24. What do raise, bare raise, and raise X from e do? raise E(...)** throws a new exception. Bare **raise** inside **except** re-raises the current one unchanged. **raise X from e** throws **X** and records **e** as its **__cause__** (exception chaining).
- C25. Why create custom exceptions and how?** To give your code meaningful, catchable error types and to carry extra data. Subclass **Exception** (usually via your own base class such as **AppError**).
- C26. Why is a bare except: bad? What should you catch?** It catches everything, including **KeyboardInterrupt**, **SystemExit** and your own typos, hiding real bugs. Catch the **specific** exceptions you can actually handle.
- C27. EAFP vs LBYL?** EAFP: "Easier to Ask Forgiveness than Permission": just try and handle the exception. LBYL: "Look Before You Leap": check conditions first. Python style often favours EAFP.
- C28. Class vs object?** A class is the blueprint (defines attributes and methods); an object is a concrete instance created from it with its own state.
- C29. What is self?** The first parameter of instance methods; it refers to the object the method was called on. It is a convention, not a keyword. **obj.m()** is **Cls.m(obj)**.
- C30. __init__ vs __new__? __new__(cls)** creates the object and returns it; **__init__(self)** initializes the already-created object and returns **None**. **__new__** is rarely overridden.
- C31. Instance variable vs class variable?** Instance variables belong to one object (**self.x**). Class variables live on the class and are shared by all instances. Assigning **obj.x = ...** creates an instance attribute that shadows the class one.
- C32. Instance vs class vs static method?** Instance method gets **self**; class method (**@classmethod**) gets **cls** and is used for factories; static method (**@staticmethod**) gets neither and is just a namespaced function.
- C33. How does Python do encapsulation?** By convention and tools, not enforcement: **__x** means internal, **__x** is name-mangled to **_Class__x**, and **@property** gives controlled access with validation.
- C34. What is abstraction and how do you enforce it?** Exposing what an object does while hiding how. Use **abc.ABC** with **@abstractmethod**: a class with unimplemented abstract methods cannot be instantiated.
- C35. What is inheritance and what does super() do?** A child class reuses and extends a parent (is-a). **super()** returns a proxy that delegates to the **next class in the MRO**, commonly the parent, e.g. **super().__init__(...)**.
- C36. Polymorphism, duck typing, method overriding?** Polymorphism: one interface, many behaviours. Duck typing: what matters is having the needed methods, not the class. Overriding: a subclass redefines an inherited method.
- C37. Does Python support method overloading?** Not directly: a later **def** with the same name replaces the earlier. Emulate with default args, ***args**, **isinstance** checks, **singledispatch**, or classmethod constructors. Operator overloading via dunder is supported.
- C38. Composition vs inheritance?** Inheritance models **is-a**; composition models **has-a** by holding other objects. Prefer composition when the relationship is not truly is-a: it is more flexible and avoids deep hierarchies.
- C39. What is the MRO and what problem does multiple inheritance create?** The MRO is the order in which classes are searched for attributes/methods (C3 linearization; see **Cls.__mro__**). Multiple inheritance can create ambiguity (the diamond problem); C3 gives a consistent, predictable order or raises **TypeError**.
- C40. What are dunder methods and how are they used?** Special methods with double underscores (**__init__**, **__str__**, **__len__**, **__eq__**, **__add__**, ...). Python calls them implicitly for syntax and built-ins (**print**, **len**, **+**, **==**), letting your classes act like built-in types.

9B. Output Answers

Q	Output	Why
O1	[1, 2, 3, 4]	b = a is an alias; append mutates the one shared list.
O2	[1, 2, 3] [1, 2, 3, 4]	Slicing makes a new list; the append affects only b .
O3	[[1, 9], [2]]	Shallow copy: the inner list [1] is shared.
O4	[[1], [2]] [[1, 9], [2]]	Deep copy duplicated the inner lists too.
O5	[[1, 0, 0], [1, 0, 0], [1, 0, 0]]	All three rows are the same list object.
O6	[[1, 0, 0], [0, 0, 0], [0, 0, 0]]	The comprehension builds three independent rows.
O7	[1, 1]	The function mutates the caller's list each time.
O8	[]	lst + [1] makes a new list bound to the local name; x is untouched.
O9	[1]	+= on a list is in-place (__iadd__), so the caller sees it.
O10	[1] [1, 2] [3]	The default list persists across calls; passing a fresh list avoids it.
O11	10	Assignment inside f creates a local x ; the global is unchanged.
O12	Raises UnboundLocalError	The assignment makes x local for the whole function, so the print reads it before assignment.
O13	1 2 3	nonlocal lets inner rebind the enclosing c ; the closure keeps its state.
O14	[2, 2, 2]	Late binding: every lambda reads i after the loop ended (i = 2).
O15	[0, 1, 2]	The default argument captures the current i at definition time.
O16	([1, 2, 3],)	The tuple still holds the same list; the list itself was mutated.
O17	error ([1, 2, 3],)	The list is extended in place first, then storing back into the tuple fails.
O18	abcd abc	Strings are immutable: += creates a new string and rebinds s only.
O19	1 [2, 3] []	list(it) consumes the rest; afterwards the iterator is exhausted.
O20	1 2 end	The default value of next is returned instead of raising StopIteration .
O21	[0, 1, 2] [0, 1, 2]	range is an iterable, not an iterator, so it can be reused.
O22	['1', '2'] []	map returns an iterator; the second list finds it empty.
O23	start A 1 B 2	The body runs only on next() , pausing at each yield .
O24	[1]	The return value goes into StopIteration.value , not into the list.
O25	5 0	The generator is exhausted after the first sum (0 + 1 + 4).
O26	[0, 10, 20]	x is looked up when the generator runs, and by then it is 10.
O27	[1, 2, 3]	yield from delegates to another iterable, then the generator continues.
O28	finally try	finally runs before the function actually returns; the print of the result comes after.
O29	2	A return in finally overrides the earlier one.
O30	A C	An exception occurred, so except and finally run; else is skipped.
O31	ok B C	No exception: try , then else , then finally .
O32	0 1 1	self.n += 1 reads the class value 0 and creates an instance attribute 1 on each object.
O33	[1]	items is one shared class-level list.

Q	Output	Why
O34	A B	<code>super().show()</code> runs the parent's version first.
O35	D B C A ['D', 'B', 'C', 'A', 'object']	<code>super()</code> follows the MRO D, B, C, A, object; each class runs once.
O36	point [P(1)] P(1)	<code>print</code> uses <code>__str__</code> ; containers and <code>repr</code> use <code>__repr__</code> .
O37	True	<code>+</code> calls <code>__add__</code> giving <code>V(3)</code> ; <code>==</code> calls <code>__eq__</code> comparing <code>a</code> .
O38	False 0	With no <code>__bool__</code> , truthiness falls back to <code>__len__</code> ; length 0 is falsy.
O39	False True	Name mangling stores the attribute as <code>_A_x</code> .
O40	two	The second <code>def f</code> replaced the first; Python has no overloading.

TIP: Every output above was produced by actually running the code

Each snippet was executed with Python during the making of these notes, and the output shown is the real output. Run them yourself and change one line at a time to see what breaks.

9C. Interview Answers

11. Why do `a = b = []` and `a = []; b = []` behave differently?

The first creates **one** list with two names; the second creates **two** lists. Mutating through one name is visible through the other only in the first case.

```
a = b = []
a.append(1)
print(b)          # [1]
c = []; d = []
c.append(1)
print(d)          # []
```

12. Create a 3x4 grid of zeros safely and explain why `[[0]*4]*3` is wrong.

Use `[[0] * 4 for _ in range(3)]`. `[[0]*4]*3` repeats the reference to one inner list, so a write to one row appears in all rows. `[0]*4` itself is safe because ints are immutable.

13. A tuple is immutable, yet `t[0].append(1)` works when `t = ([], )`. Explain.

Immutability applies to the tuple's **references**: it always points to the same list. The list is a separate mutable object and can change. This is why such a tuple is also **not hashable**, since hashing needs hashing its contents.

14. What happens with `t = ([1, 2],)`; `t[0] += [3]`?

It raises **TypeError** **but the list still becomes `[1, 2, 3]`**. Augmented assignment does two steps: `t[0].__iadd__([3])` (mutates the list, succeeds) then `t[0] = result` (tuple item assignment, fails).

15. Write your own deep copy for nested lists/dicts. What does `copy.deepcopy` add?

deepcopy also handles tuples/sets/custom objects (via `__deepcopy__`), and uses a **memo dict** so shared references stay shared and **cycles** do not recurse forever. The toy version below does not.

```
def deep(x):
    if isinstance(x, list):
        return [deep(i) for i in x]
    if isinstance(x, dict):
        return {k: deep(v) for k, v in x.items()}
    return x          # immutables returned as-is
```

16. Why is `x is None` preferred over `x == None`?

None is a singleton, so identity is exact and fast. `==` can be overloaded by `__eq__` (NumPy arrays, custom classes) and may return something unexpected, and `is` cannot be overloaded.

17. What is small-integer caching and why should you not rely on it?

CPython pre-creates ints from -5 to 256 and reuses them, so `is` may be True for equal small ints. It is an **implementation detail** that varies with version, context and compile-time constant folding. Always compare values with `==`.

I8. `b = a; a += [4]` versus `b = a; a = a + [4]`: difference?

`a += [4]` mutates the list in place, so `b` also sees `[..., 4]`. `a = a + [4]` builds a new list and rebinds `a`; `b` keeps the old list.

```
a = [1]; b = a; a += [4];    print(b)    # [1, 4]
a = [1]; b = a; a = a + [4]; print(b)    # [1]
```

I9. How does Python manage memory?

Objects live on a private heap. CPython uses **reference counting** (freed at zero) plus a **cyclic garbage collector** for reference cycles. `del` removes a name, not necessarily the object. `gc` and `sys.getrefcount` let you inspect it.

I10. Explain the mutable default argument bug, when the default is evaluated, and the fix.

Defaults are evaluated **once at function definition** and stored in `f.__defaults__`, so a mutable default persists across calls. Fix: default to `None` and create the object inside. It can be used deliberately as a cache.

```
def f(x, acc=None):
    if acc is None:
        acc = []
    acc.append(x)
    return acc
```

I11. Why does `[lambda: i for i in range(3)]` return `[2, 2, 2]` and how do you fix it?

Closures capture the **variable**, not its value (late binding). All lambdas read `i` after the loop finishes (2). Fix by binding at definition: `lambda i=i: i`, or use `functools.partial`.

I12. `global` vs `nonlocal`, and why does `UnboundLocalError` happen?

`global` rebinds a module-level name; `nonlocal` rebinds a name in the nearest enclosing function. If a function assigns to a name anywhere, that name is **local throughout** the function, so reading it before the assignment raises `UnboundLocalError`.

I13. Why is removing items from a list while looping over it buggy, and what is the correct way?

The loop uses an internal index; removals shift later items left so some are skipped. Build a new list (`[x for x in a if keep(x)]`), iterate over a copy (`for x in a[:]`), or loop backwards / use `filter`.

I14. Iterable vs iterator: is a file an iterator? Is `range`?

A file object **is** an iterator (yields lines, `iter(f)` is `f`). `range` is an **iterable** (a lazy, re-usable sequence), not an iterator: `next(range(3))` raises `TypeError`.

I15. Implement an iterator class that yields Fibonacci numbers up to a limit.

Iterator protocol: `__iter__` returns self, `__next__` returns the value or raises `StopIteration`.

```
class FibUpTo:
    def __init__(self, limit):
        self.limit = limit
        self.a, self.b = 0, 1
    def __iter__(self):
        return self
    def __next__(self):
        if self.a > self.limit:
            raise StopIteration
        value = self.a
        self.a, self.b = self.b, self.a + self.b
        return value

print(list(FibUpTo(20)))    # [0, 1, 1, 2, 3, 5, 8, 13]
```

I16. Why does changing a dict during iteration raise `RuntimeError`, and how do you avoid it?

The dict detects that its size changed during iteration and refuses to continue, since the internal order/positions may be invalid. Iterate over a snapshot: `for k in list(d):` or build a new dict with a comprehension.

I17. What does the two-argument form `iter(callable, sentinel)` do?

It calls `callable()` repeatedly and yields each result until the result equals `sentinel`, then stops. Classic use: `for block in iter(lambda: f.read(1024), b'')`.

I18. Generator vs list: trade-offs?

Generator: $O(1)$ memory, lazy, can be infinite, starts instantly, but one-shot with no indexing/**len**. List: $O(n)$ memory, computed up front, supports indexing, slicing, repeat traversal. Choose by data size and access pattern.

I19. What exactly happens when you call a generator function?

Nothing in the body runs. Python creates and returns a **generator object** holding the function's frame. The body runs only when you call **next()** (or **iterate**), up to the first **yield**.

I20. Can a generator use return? What happens to the value?

Yes. **return v** ends the generator by raising **StopIteration(v)**. **for** loops ignore **v**, but **yield from** evaluates to it, and you can read it from **e.value** when catching **StopIteration** manually.

I21. What does yield from do?

It delegates to a sub-iterable/sub-generator: yields all its items and evaluates to its return value. It replaces **for x in sub: yield x** and also forwards **send()/throw()**.

I22. Sum the squares of the first million integers without building a list.

Use a generator expression so values are produced and consumed one at a time.

```
total = sum(i * i for i in range(1, 1_000_001))
```

I23. A generator is exhausted. How can you traverse the data again?

Recreate the generator (call the function again), store results in a list, or use **itertools.tee(gen, n)** to split into independent iterators (which buffers items in memory).

I24. What does send() do on a generator?

It resumes the generator and makes the paused **yield** expression evaluate to the sent value (**x = yield**). The first call must be **next(g)** or **g.send(None)** to reach the first **yield**. This is the basis of coroutine-style generators.

I25. Trace: try has a return, finally prints. In what order do things happen?

The return value is evaluated, then **finally** runs, then the function actually returns. If **finally** itself contains **return**, that value replaces the earlier one and any pending exception is discarded.

I26. Why put code in else instead of at the end of try?

It documents that this code runs only on success and, importantly, exceptions raised there are **not** caught by the preceding **except**. So **except** guards only the line that can fail.

I27. Why does except Exception not catch Ctrl+C?

KeyboardInterrupt, **SystemExit** and **GeneratorExit** derive from **BaseException**, not **Exception**. That lets programs be interrupted or exit even inside broad handlers.

I28. raise vs raise X from e; what happens with a bare raise outside an except?

Inside **except**, bare **raise** re-raises the active exception with the original traceback. **raise X from e** sets **X.__cause__ = e** for chaining. A bare **raise** with no active exception gives **RuntimeError: No active exception to reraise**.

I29. Design a small custom exception hierarchy for a banking app.

Have one application base class so callers can catch the whole family or a specific member.

```
class BankError(Exception): pass
class InsufficientFunds(BankError): pass
class AccountLocked(BankError): pass

try:
    raise InsufficientFunds("need 30 more")
except BankError as e:
    print(type(e).__name__, e)  # InsufficientFunds need 30 more
```

I30. Difference between __init__ and __new__? Implement a singleton.

__new__ creates and returns the instance (called first, receives **cls**); **__init__** initializes it. A singleton overrides **__new__** to return one cached instance.

```
class Singleton:
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
        return cls._inst

print(Singleton() is Singleton())    # True
```

I31. Class variables vs instance variables: what is the `self.count += 1` trap?

If **count** exists only on the class, `self.count += 1` reads the class value then **creates an instance attribute**, leaving the class variable unchanged. Modify shared state through the class name (`cls.count += 1`).

I32. When would you use `@classmethod` versus `@staticmethod`?

@classmethod when you need the class (**cls**): alternative constructors (**from_string**), factories that must work with subclasses. **@staticmethod** for a helper that needs neither the class nor the instance but belongs in the class namespace.

I33. Is Python encapsulation "real"? Explain name mangling.

No true private members. `_x` is a convention; `__x` is renamed to `_Class__x` to prevent accidental override in subclasses, but is still accessible. Python trusts programmers ("we are all consenting adults").

I34. What is `@property` and why use it?

It turns a method into an attribute-style accessor (with optional setter/deleter). You can add validation or computed values later without breaking callers that use plain attribute syntax.

I35. How would you achieve method overloading in Python?

Default arguments, `*args/**kwargs`, `isinstance` checks, `functools.singledispatch/singledispatchmethod`, or `classmethod` alternate constructors. Defining the same method name twice just overwrites the first.

I36. Explain the MRO for a diamond, and what `super()` really does.

For `class D(B, C)` with `B(A)`, `C(A)` the MRO is **D, B, C, A, object** (C3). `super()` moves to the **next class in the instance's MRO**, not simply the parent, so **B's `super()` calls C**. Inspect with `D.__mro__`.

I37. Why should every class in a multiple-inheritance chain call `super().__init__()`?

Cooperative inheritance: each `__init__` calls the next one in the MRO so every base gets initialized exactly once. If one class omits `super()`, later classes in the MRO are skipped and their attributes are never set.

I38. What happens to `__hash__` when you define `__eq__`? Why does it matter?

Python sets `__hash__ = None`, making instances unhashable (no dict keys/set members) unless you also define `__hash__`. Objects that compare equal must have equal hashes, and the hash fields should be immutable.

I39. `__str__` vs `__repr__`: which is used where?

`print/str/f-strings` use `__str__` (falls back to `__repr__`). The REPL, `repr()`, and items inside containers use `__repr__`. Convention: `__repr__` is unambiguous (ideally `cls(args)`), `__str__` is readable.

I40. Composition over inheritance: what does it mean? `isinstance` vs `type()`?

Prefer building objects from collaborating parts (has-a) over deep is-a hierarchies; it reduces coupling and lets behaviour be swapped at runtime. Use inheritance only for genuine is-a with substitutability. `isinstance(x, C)` accepts subclasses (preferred); `type(x) == C` is exact-match only.

10. Common Traps

#	Trap	What actually happens	Fix / rule
1	b = a for lists/dicts	Alias, not a copy	a.copy() , a[:] , copy.deepcopy(a)
2	a.copy() on nested data	Inner objects shared	copy.deepcopy(a)
3	[[0]*3]*3	Three references to one row	[[0]*3 for _ in range(3)]
4	def f(x, L=[])	Default created once, shared	L=None , create inside
5	lst += [1] inside a function	Mutates the caller's list	Know += is in-place; use lst = lst + [1] to avoid
6	lst = ... inside a function	Rebinding: caller unchanged	Return the new value
7	t[0] += x on a tuple holding a list	TypeError and the list changes	Do not mutate through tuple items
8	is on numbers/strings	Works sometimes (caching)	== for values, is only for None/True/False
9	Read then assign a global in a function	UnboundLocalError	global x , or pass/return values
10	lambda: i in a loop	All see the last i	lambda i=i: i
11	Naming a variable list , sum , id	Shadows the built-in	Pick another name
12	Iterating a list while removing	Items skipped	Iterate a copy or build a new list
13	Changing dict size in a loop	RuntimeError	for k in list(d):
14	Reusing an iterator / generator	Second pass is empty	Recreate it or store in a list
15	len(gen) or gen[0]	TypeError	Convert to list, or next(gen)
16	Generator expression reads variables late	Uses the value at run time	Consume immediately or bind values
17	Bare except: / except Exception: pass	Hides real bugs	Catch specific errors, log or re-raise
18	General except before specific	Specific clause unreachable	Order: specific first
19	return inside finally	Overrides result, swallows exception	Avoid
20	Using e after the except block	NameError (deleted)	Copy it to another name
21	Mutable class variable (items = [])	Shared by all instances	Create in __init__
22	self.count += 1 with class variable	Creates instance attribute	cls.count += 1
23	Forgot super().__init__()	Parent attributes missing	Call it (and in every cooperative class)
24	Two methods with one name	Later replaces earlier	Default args / singledispatch
25	__eq__ without __hash__	Instances unhashable	Define both, hash immutable fields
26	Only __str__ defined	repr() /lists show <Cls object at ...>	Always define __repr__
27	class X(A, B) where B subclasses A	Inconsistent MRO TypeError	Subclasses before bases: X(B, A)
28	super() assumed to be "the parent"	It is the next class in the MRO	Trace __mro__
29	Bare raise outside except	RuntimeError	Use inside a handler
30	Expecting __x to be private	Only mangled to _cls_x	Use _x convention

11. Revision Sheet

One page per topic, in the order to recall them. Cover the right column and explain each line aloud.

References & memory

- Variables are **names**; assignment **binds**, never copies.
- Object = identity + type + value. `==` value, `is` identity, `id()` identity number.
- **Mutating** changes the object (all names see it); **rebinding** moves one name.
- Immutable: int, float, bool, str, tuple, frozenset, bytes, None. Mutable: list, dict, set, most objects.
- Copies: `b=a` alias; `a[:]`, `a.copy()`, `list(a)`, `copy.copy` shallow; `copy.deepcopy` deep.
- `[x]*n` repeats a **reference**. Build grids with a comprehension.
- Arguments: pass by object reference. Mutation visible, rebinding not. `+=` on a list is mutation.
- Mutable default: created once at **def**. Use **None**.
- Scope: LEGB. Assigning makes a name local for the whole function. **global**, **nonlocal**. Closures late-bind.

Iterables, iterators, generators

- Iterable: `__iter__`. Iterator: `__next__` + `__iter__` returns self. **StopIteration** ends it.
- **for** = `iter()` once, then `next()` until **StopIteration**.
- Iterators are **one-shot**. Lists/range/str/dict/set are re-iterable; generators, files, map, zip, filter, enumerate are not.
- Generator function: has **yield**; call returns generator; state preserved between `next()` calls.
- Generator expression (`...`): lazy, O(1) memory, no `len`/index. **yield from** delegates; **return v** -> **StopIteration.value**.
- **yield** pauses, **return** ends.

Exceptions

- **try** risky | **except** handle | **else** success only | **finally** always.
- First matching **except** wins; specific before general; tuple for several types.
- **raise E**, bare **raise** re-raises, **raise X from e** chains. Custom: subclass **Exception**.
- Never bare **except**; keep **try** small; use **with** for cleanup; **finally** return overrides.
- **Exception** does not include **KeyboardInterrupt/SystemExit (BaseException)**.

OOP

- Class = blueprint; object = instance. **self** = the instance. `__init__` initializes, `__new__` creates.
- Instance var per object; class var shared; assigning via instance shadows it.
- Methods: instance (**self**), `@classmethod` (**cls**, factories), `@staticmethod` (neither).
- Encapsulation: `_x` convention, `__x` mangled, `@property`. Abstraction: **ABC** + `@abstractmethod`.
- Inheritance = is-a; override; **super()** = next in MRO. Composition = has-a.
- No true overloading: later **def** wins. Use defaults, ***args**, **singledispatch**, classmethods.
- MRO via C3: children before parents, base order kept. **D(B, C)** -> D, B, C, A, object.
- Dunders: `__repr__` (always), `__str__`, `__len__`, `__eq__` (+ `__hash__`), `__add__`, `__lt__`, `__getitem__`, `__bool__` -> falls back to `__len__`.

30-second self-test

Say the answer aloud	Answer
<code>a=[1]; b=a; b+=[2]; print(a)</code>	<code>[1, 2]</code>
<code>def f(x=[]): x.append(1); return x</code> called twice	<code>[1]</code> then <code>[1, 1]</code>
<code>g=(i for i in range(3)); list(g); list(g)</code>	<code>[0, 1, 2]</code> then <code>[]</code>
<code>try: return 1/finally: print("f")</code>	prints <code>f</code> , returns <code>1</code>
MRO of <code>class D(B, C), B(A), C(A)</code>	<code>D, B, C, A, object</code>
<code>class X: __eq__</code> only. Is <code>{X()}</code> allowed?	No: unhashable